

IF231303 Arsitektur Perangkat Lunak

Pertemuan 02: Layered Architecture

September 6, 2026

Alfa Yohannis



Tujuan Pembelajaran

- 1 Menjelaskan pembagian tanggung jawab keempat *layer* beserta arah ketergantungannya.
- 2 Membedakan *closed layer* dari *open layer*, lalu menentukan kapan masing-masing layak dipilih.
- 3 Mengukur kedalaman *call stack*, *layer violation*, dan selisih waktu antara kedua bentuk.

Pendahuluan

- *Layered architecture* menata aplikasi menjadi beberapa *layer* yang bertumpuk.
- Setiap *layer* mengerjakan satu jenis urusan teknis, lalu memanggil *layer* di bawahnya.
- Bentuk ini menjadi pilihan pertama bagi sebagian besar sistem, dan sering dipakai tanpa pertimbangan panjang.

Alasan pemakaiannya sering bersifat organisasi, bukan teknis. Pembagian *layer* mengikuti pembagian kerja yang sudah ada di banyak tim, yaitu ada yang mengerjakan antarmuka, ada yang mengerjakan aturan bisnis, dan ada yang mengerjakan basis data.

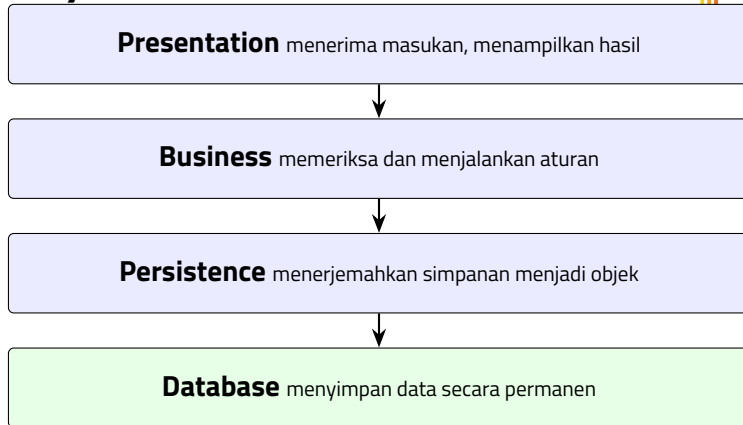
Kemudahan itu ada harganya. Ketika aplikasi membesar, satu fitur baru sering menyentuh keempat *layer* sekaligus.

Contoh Isi Setiap *Layer*

<i>Layer</i>	Contoh isinya	Kalimat kebutuhan
Presentation	Menyusun teks 100 USD = 1.625.000 IDR	Hasil ditampilkan dengan pemisah ribuan
Business	Menolak nominal nol, memeriksa token dan peran	Hanya staf boleh melihat kurs jual
Persistence	Mengubah baris tabel menjadi tupel	Kurs disimpan per pasangan mata uang
Database	Tabel kurs beserta kolom margin	Margin keuntungan disimpan terpisah dari nilai dasar

Uji cepat: bila sebuah kebutuhan tetap berlaku walaupun layar diganti menjadi API, kebutuhan itu milik *business layer*.

Empat *Layer* Baku



Panggilan hanya boleh mengarah ke bawah, dan tidak boleh melompat.

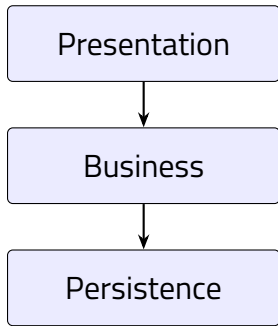
Dua Konsekuensi Aturan Tersebut

- Satu *layer* dapat diganti tanpa menyentuh *layer* lain, selama bentuk panggilannya tetap. Jadi, tidak perlu mengubah *layer* lain.
- Permintaan sederhana tetap harus melewati seluruh *layer*, walaupun sebagian *layer* tidak mengerjakan apa pun.

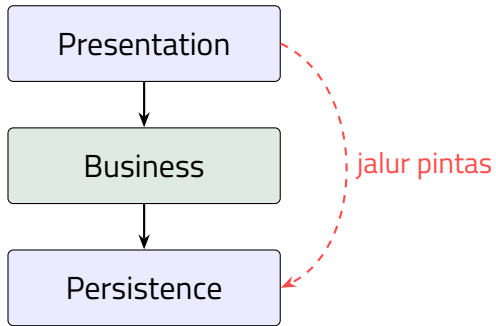
Konsekuensi kedua menimbulkan pertanyaan berikutnya, yaitu apakah setiap permintaan benar-benar wajib melewati semua *layer*.

Closed Layer dan Open Layer

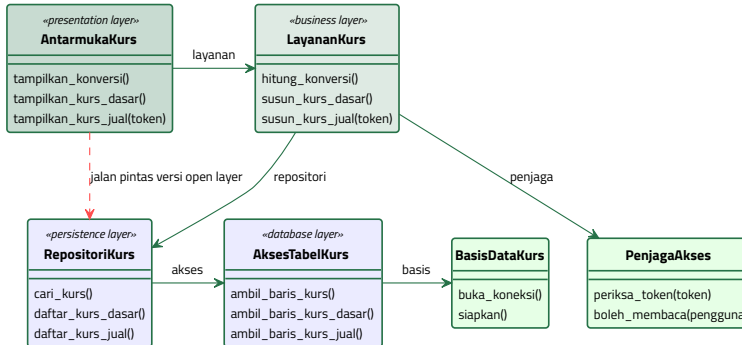
Closed



Open



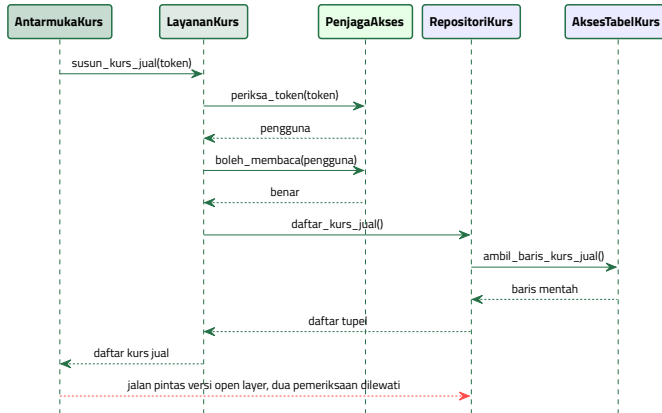
Empat *Layer* sebagai Empat Kelas



Kolaborator diterima lewat konstruktor, sehingga arah ketergantungannya terbaca dari tanda tangan konstruktornya.

PenjagaAkses dan BasisDataKurs bukan *layer*, melainkan kolaborator di luar susunan itu.

Urutan Panggilan Kurs Jual



Dua panah ke PenjagaAkses adalah pemeriksaan yang dituntut *business layer*. Panah putus-putus paling bawah memotong tepat di situ.

Architecture Sinkhole

- Permintaan mengalir melewati beberapa *layer* tanpa satu pun *layer* mengerjakan sesuatu.
- Keadaan ini hampir tidak mungkin dihapus, sehingga yang diusahakan adalah menekan jumlahnya.
- Yang diperiksa adalah apakah *business layer* lebih sering meneruskan panggilan daripada mengerjakan sesuatu.
- *Layer* mana yang tertutup dan mana yang terbuka wajib dicatat dan disepakati satu tim.

Pass-through yang Wajar dan yang Tidak

```
def susun_kurs_dasar(self):  
    # Kurs dasar diterbitkan untuk umum, meniru JISDOR.  
    # Tidak ada tuntutan bisnis, sehingga murni pass-through.  
    return self.repositori.daftar_kurs_dasar()  
  
def susun_kurs_jual(self, token):  
    # Kurs jual memuat margin keuntungan bank, dan besar  
    # keuntungan itu komersial, sehingga wajib diperiksa.  
    pengguna = self.penjaga.periksa_token(token)  
    if pengguna is None:  
        raise PermissionError("Token tidak dikenali")  
    if not self.penjaga.boleh_membaca(pengguna):  
        raise PermissionError("Peran tidak berhak membaca")  
    return self.repositori.daftar_kurs_jual()
```

Keduanya metode pada LayananKurs. Jalan pintas pada yang pertama wajar. Pada yang kedua, dua pemeriksaan ikut hilang.

Analogi: Tiga Meja dan Satu Cap

- Surat pengantar harus melewati tiga meja sebelum disetujui.
- Meja pertama membaca isinya lalu memutuskan.
- Meja kedua dan ketiga hanya membubuhkan cap, lalu meneruskan.
- Ketiganya menambah waktu, tetapi hanya satu menambah nilai.

Prosedurnya terlihat rapi di atas kertas, tetapi dua meja terakhir adalah *sinkhole*. Itulah yang terjadi pada *layer* yang hanya meneruskan panggilan.

Kasus: Jalan Pintas yang Dipakai Ulang

- 1 Tim membuat halaman perbandingan kurs yang terbuka untuk pengunjung.
- 2 Agar cepat selesai, halaman itu langsung membaca *persistence layer*.
- 3 Waktu itu pilihannya masih aman, sebab yang tampil hanya kurs dasar.
- 4 Beberapa bulan berikutnya, kurs jual diminta ikut tampil di halaman yang sama.
- 5 Nama metodenya tinggal ditukar, sebab jalannya sudah terbuka sejak awal.

Sejak saat itu margin keuntungan bank ikut terbaca siapa saja. Tidak ada pesan galat dan tidak ada yang gagal, sehingga tidak ada yang curiga.

Hasil Pengukuran

Berkas	Metode <i>pass-through</i>	<i>Layer violation</i>
aplikasi_tertutup	1 dari 12, yaitu 8,3 persen	0
aplikasi_terbuka	0 dari 6	2

Kurs jual, seratus putaran	Nilai
Median jalur <i>closed</i> , dengan pemeriksaan	0,7308 ms
Median jalur <i>open</i> , tanpa pemeriksaan	0,7188 ms
Selisih median antar jalur	-0,0121 ms
Putaran jalur <i>open</i> lebih cepat	48 dari 100

Angka 48 dibandingkan dengan 50, yaitu nilai yang muncul bila kedua jalur sama cepat.

Selisih Dibanding *Range*

Range di dalam satu jalur *closed*: 0,7754 ms



Selisih median antar kedua jalur: 0,0121 ms

← 64 kali lebih kecil

Dua putaran pada jalur yang sama saja dapat berselisih 0,7754 ms, sedangkan selisih antar kedua jalur hanya 0,0121 ms. Selisih yang dicari tenggelam di dalam *range* pengukurannya sendiri.

- Empat *layer*, satu aturan, yaitu panggilan hanya mengarah ke bawah.
- Aturan itu menahan perubahan di satu *layer*, tetapi memaksa permintaan sederhana melewati *layer* yang tidak bekerja.
- Metode *pass-through* ditekan jumlahnya, bukan dihapus sepenuhnya.
- *Open layer* memangkas kode *pass-through*, tetapi menambah dua *layer violation*.
- Jalan pintas hanya lebih cepat pada 48 dari 100 putaran, dan selisihnya 64 kali lebih kecil daripada *range* pengukurannya.
- Yang menentukan bukan kecepatan, melainkan ada tidaknya tuntutan bisnis yang ikut dilewati.

Persiapan Latihan

```
# sekali saja, dari akar repositori
bash sources/siapkan_venv.sh
source .venv/bin/activate

cd sources/chapter02
python basis_data.py
```

Venv dibuat sekali di `.venv`, lalu cukup diaktifkan pada terminal berikutnya. Satu basis kode, tiga masalah berbeda. Aplikasi melayani dua permintaan pada tabel yang sama, yaitu kurs dasar yang publik dan kurs jual yang memuat margin.

Latihan 1: Menelusuri Jalan Permintaan

```
# closed, aplikasi_tertutup.py
antarmuka.tampilkan_kurs_jual(token) ----> koneksi.execute(KUERI_KURS_JUAL)
antarmuka.tampilkan_kurs_dasar()      ----> koneksi.execute(KUERI_KURS_DASAR)

# open, aplikasi_terbuka.py
antarmuka.tampilkan_kurs_jual()        ----> koneksi.execute(KUERI_KURS_JUAL)
antarmuka.tampilkan_kurs_dasar()      ----> koneksi.execute(KUERI_KURS_DASAR)
```

- 1 Lengkapi isi setiap panah dengan membaca kode, tanpa menjalankannya.
- 2 Hitung metode di antara kedua ujung, pemeriksaannya, dan metode *pass-through*-nya.
- 3 Bandingkan versi *closed* dengan versi *open* pada permintaan yang sama.

Pertanyaan: kedua versi berselisih tepat satu metode. Metode mana yang hilang, dan apa yang dibawahanya?

Latihan 2: Menemukan *Layer Violation*

- 1 Salin konstanta LAPISAN pada kedua berkas, lalu baca konstruktor setiap kelas beserta anotasi tipenya.
- 2 Gambarkan panah dari kelas pemegang ke kelas yang dipegang, lalu tandai panah yang melompati satu *layer* atau lebih.
- 3 Jalankan kedua aplikasi, lalu bandingkan perlakuannya terhadap token.

Pertanyaan: bagian kode mana yang tidak melewati *business layer* melainkan langsung menuju sumber datanya? Sebutkan kelas, metode, dan baris rujukannya.

Latihan 2: Akibat pada Ketiga Token

```
python aplikasi_tertutup.py dasar
# EUR -> IDR: 17600.0      <- tanpa token, boleh
python aplikasi_tertutup.py jual token-andi
# EUR -> IDR: 17690.0      <- dasar + margin 90
python aplikasi_tertutup.py jual token-budi
# ditolak: Peran tamu tidak berhak membaca
python aplikasi_tertutup.py jual token-palsu
# ditolak: Token tidak dikenali
python aplikasi_terbuka.py
# jual : EUR -> IDR: 17690.0 <- bermargin, tanpa token
```

Token Budi lolos autentikasi tetapi gagal otorisasi, sedangkan token palsu gagal sejak autentikasi. Keduanya tidak pernah berjalan pada versi *open*.

Latihan 3: Mengukur Biaya Pemeriksaan

python ukur_lapisan.py

- 1 Perhatikan bahwa skrip menjalankan seratus putaran, bukan satu.
- 2 Perhatikan baris kemajuan yang dicetak setiap sepuluh putaran, lalu catat kedelapan angka pada laporan akhirnya.
- 3 Baca baris *Range* dibagi selisih, yaitu berapa kali *range* di dalam satu jalur lebih besar daripada selisih antar kedua jalur.
- 4 Catat pada berapa putaran jalur *open* lebih cepat, lalu bandingkan dengan lima puluh, yaitu nilai bila kedua jalur sama cepat.
- 5 Jalankan sekali lagi, lalu bandingkan kedua hasilnya.

Pertanyaan: hitung penghematannya dalam persen, lalu bandingkan dengan tuntutan bisnis yang dilepas.